INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

COURSE SYLLABUS

Course Information	Details
Semester	3rd Semester
ISCED Code	0613
Course Code	CSE-2340
Course Title	Software Development 1
Credit Hours	2
Contact Hours	4 laboratory classes per week
Type	Core, Engineering
Prerequisite	CSE-1222 (Computer Programming 2 Lab)
Co-requisite	CSE-1221 (Computer Programming 2)
Delivery	Laboratory / Practical
Course Structure	15 teaching weeks × 4 classes; separate 100-minute mid-term practical slot

Course Assessments

Assessment Component	Marks	Assessment Format
Attendance	10	Continuous
Mid-term In-lab Practical	20	Separate 100-minute practical assessment
Six Mini Projects / Continuous Assessment	40	Individual project + dedicated evaluation class
Final Group Project / Showcase	30	Team project + individual viva
Total	100	

Course Rationale

This course provides a practical foundation in modern front-end development. Students progress from web architecture and semantic HTML through CSS, responsive design, JavaScript, DOM manipulation, asynchronous APIs, React, routing, ShadCN/ui, Firebase Authentication, Firestore, Git/GitHub workflows, and deployment. The revised workload deliberately protects the core skills required to become a capable entry-level front-end developer while compressing topics that are useful but not essential at this stage. The course also reserves one complete 50-minute class for the evaluation of each of the six individual mini projects because the class may contain a large number of students and meaningful individual verification cannot be reliably completed during normal teaching time.

Objectives

1. Apply modern front-end development principles and practices to build practical web interfaces and applications.
2. Analyze interface structure, responsive design decisions, client-side behavior, accessibility, and common implementation choices.
3. Use HTML, CSS, modern JavaScript, DOM APIs, asynchronous APIs, React, routing, component libraries, authentication, persistence, Git/GitHub, and deployment tools.
4. Develop and integrate a responsive interactive front-end application using modern development workflows.
5. Work effectively as an individual and team member, communicate technical work, review implementations, and demonstrate project ownership.

Mapping of CLO–PLO–DL–KP–EP–EA

CLO	PLO	DL	KP	EP	EA	Teaching–Learning Strategy	Assessment Strategy
CLO1	PO1	C3	K3	P1	–	Laboratory demonstration, guided practice, code-along, notes	Mini projects, class performance, mid-term practical
CLO2	PO3	C4	K3	P1	–	Demonstration, guided analysis, DevTools practice, studio critique	Mini projects, mid-term practical, project evaluation
CLO3	PO5	C3	K5, K6	P1	–	Live build, code-along, supervised build/debug, practical lab work	Mini projects, practical evaluation
CLO4	PO3, PO5	C6	K5, K6	P1	–	Integration studio, guided project development, debugging	MP4–MP6, final group project, viva
CLO5	PO9, PO10	C5 / A3	K5	P2	–	Peer review, code review, demonstration, presentation	Project evaluation, PR review, final presentation, individual viva

Revision principle

The course is now organized around one question: "What does a third-semester student actually need to become a functional front-end developer?" Core foundations remain. Specialist, project-specific, or theory-heavy material is reduced or removed. The saved time is explicitly converted into assessment time rather than simply adding more content.

Revised Weekly Course Content

Week	Content / Deliverable
1	C1 — Web architecture: client/server, DNS, HTTP request/response, status codes, browser rendering, hosting C2 — Development environment: VS Code, Node/npm, Live Server, first HTML file C3 — Semantic HTML: headings, text, links, images, forms, document structure C4 — Git basics: init, add, commit, push, repository, .gitignore
2	C1 — Navigation, forms and tables: practical HTML patterns C2 — Accessibility essentials: landmarks, alt text, labels, heading order, keyboard navigation, focus and contrast C3 — CSS application methods, selectors, text properties and units C4 — Box model, margin, padding, width/height, display and backgrounds
3	C1 — Flexbox C2 — CSS Grid and responsive/mobile-first design C3 — Specificity, cascade, pseudo-classes and practical DevTools inspection; live build of a complete static page ***C4 — MINI PROJECT 1 EVALUATION: semantic HTML/CSS responsive page and deployment***
4	C1 — Design handoff using Figma or Pixso: frames, spacing, measurements and asset export (one focused practical session) C2 — CSS positioning: relative, absolute, fixed, sticky; practical z-index use C3 — Tailwind CSS v4: setup, typography, spacing, responsive flex/grid C4 — Live build: responsive Tailwind page
5	C1 — Responsive deployment, image sourcing and basic image optimization C2 — JavaScript fundamentals: variables, primitives, template literals C3 — Arrays, objects and functions; practical data handling ***C4 — MINI PROJECT 2 EVALUATION: own responsive design using Tailwind, deployed***
6	C1 — DOM and query selectors C2 — DOM content, attributes, classList, createElement and appendChild C3 — Events, addEventListener, preventDefault; practical event handling C4 — Live build: small interactive DOM application
7	C1 — Array methods for UI work: map, filter, forEach and find C2 — Supervised DOM build/debug session; Mini Project 3 preparation ***C3–C4 — MID-TERM PRACTICAL EXAMINATION: 100 minutes (20 marks)***
8	C1 — Modern JavaScript essentials: arrow functions, spread/rest and destructuring C2 — Optional chaining, object access, for-of/for-in, truthy/falsy, == vs === C3 — GitHub workflow: repositories, branches and pull requests; team formation and final project brief ***C4 — MINI PROJECT 3 EVALUATION: DOM application or debug-an-existing-codebase task***
9	C1 — HTTP/HTTPS and APIs; synchronous vs asynchronous JavaScript at a practical level C2 — Promises, async/await and try/catch/finally C3 — JSON, fetch, rendering, loading/error/empty states; Network tab C4 — *Final group project proposal review* and scope approval, *SDLC and Agile*: software development lifecycle, Agile principles, user stories, product backlog, sprint planning, implementation, testing, review, and retrospective; applying the workflow to the final group project
10	C1 — Live build: API-driven page with dynamic rendering and loading/error/empty states C2 — Practical search/filter interaction; project-specific modal behavior only where needed C3 — localStorage: primitive values, objects and simple cart/persistence pattern

	C4 — MINI PROJECT 4 EVALUATION: API-driven application with local persistence and deployment
11	C1 — Vite scaffolding, components and JSX C2 — Props, conditional rendering and lists C3 — Event handling and useState; practical state model C4 — useEffect, API calls with state and practical React project structure
12	C1 — Immutable state updates and lifting state up C2 — React Router: routes, Outlet, Link/NavLink C3 — Dynamic routes, useParams, useNavigate, 404/loading states C4 — ShadCN/ui: prepared setup and practical Button, Card, Dialog and Table components
13	C1 — Controlled React forms and practical validation C2 — Reusable components, props and children C3 — React debugging: component inspection and common state/prop problems ***C4 — MINI PROJECT 5 EVALUATION: React SPA with routing, reusable components and ShadCN/ui, deployed***
14	C1 — Firebase project and SDK; Google sign-in/sign-out C2 — Email/password registration/login, auth state and private routes (without Context API) C3 — Firestore: one collection, basic read/write; supplied security-rules starter ***C4 — MINI PROJECT 6 EVALUATION: authentication + one Firestore collection, deployed***
15	C1 — Practical project polish: responsive behavior, accessibility checks and lightweight Lighthouse audit C2 — GitHub pull-request review and final project submission workflow; no merge/merge-conflict lesson C3 — Integration studio and final debugging ***C4 — Final Group Project Showcase and Presentation: separate final showcase slot***

Dedicated Mini-Project Evaluation Plan

Each mini project receives a dedicated 50-minute evaluation class. Students submit the repository and deployed URL before the evaluation class; the class is used for rapid individual check-off, demonstration, and authenticity verification.

Project	Due / Evaluation	Deliverable	Marks	Evaluation Focus
MP1	Week 3 C4	Semantic HTML + CSS + responsive layout + deployment	5	Structure, styling, responsiveness, deployment, individual explanation
MP2	Week 5 C4	Own responsive design + Tailwind + deployment	5	Design translation, Tailwind usage, responsive behavior, repository/deployment
MP3	Week 8 C4	DOM manipulation application or debugging task	7	DOM logic, events, interaction, debugging and code ownership
MP4	Week 10 C4	API application + localStorage + loading/error states + deployment	7	fetch/API handling, UI states, persistence, debugging, deployment
MP5	Week 13 C4	React SPA + routing + reusable components + ShadCN/ui + deployment	8	Components, props/state, routing, UI library usage, deployment
MP6	Week 14 C4	Firebase Authentication + one Firestore collection + deployment	8	Authentication, private access, basic persistence, configuration safety, deployment

Recommended large-class evaluation procedure
1) Repository and deployed URL are checked before class. 2) Students are called in a fixed order. 3) Each student gives a short live demonstration of the required feature. 4) Instructor asks 1–2 ownership/debugging questions. 5) If time permits, the student makes one small modification live. 6) The rubric records the result immediately. This keeps evaluation scalable while preserving individual evidence.

Mini-Project Evaluation Rubric

Criterion	Typical Weight within Each Project	What is Checked
Core functionality	40%	Required feature works correctly.
Technical implementation	25%	Appropriate HTML/CSS/JS/React/Firebase technique is used.
Responsiveness / usability	15%	Interface behaves appropriately on different screen sizes.
Code organization	10%	Reasonable structure, naming and readability.
Individual demonstration	10%	Student can explain, modify or debug their own work.

Mid-term Practical Examination : 100 Minutes (C3–C4 Block)

• The task must be achievable within **100 minutes** and **must not** require React, Firebase, or topics scheduled after the mid-term.

• **Suggested requirements**: semantic HTML, responsive CSS, and at least one meaningful JavaScript interaction.

• **Emphasis**: implementation, responsiveness, interaction and debugging rather than memorization.

• The examination replaces the regular Week 7 C3 and C4 activities.

• **Individual** practical assessment.

• **Marks**: 20.

• **Duration**: 100 minutes (C3–C4).

The mid-term is a 20-mark individual practical examination conducted during one complete C3–C4 block in Week 7. Since each C1/C2 class is 50 minutes, C3–C4 provides the required 100 minutes without creating an additional class outside the normal weekly schedule.

Assessment Strategy

Assessment	Marks	CLO Alignment	Nature
Attendance	10	CLO5	Continuous
Mid-term Practical	20	CLO1, CLO2, CLO3	Individual, 100-minute practical
Mini Project 1	5	CLO1, CLO2, CLO3	Individual + dedicated evaluation
Mini Project 2	5	CLO2, CLO3	Individual + dedicated evaluation

Mini Project 3	7	CLO2, CLO3	Individual + dedicated evaluation
Mini Project 4	7	CLO3, CLO4	Individual + dedicated evaluation
Mini Project 5	8	CLO3, CLO4	Individual + dedicated evaluation
Mini Project 6	8	CLO4	Individual + dedicated evaluation
Final Group Project / Showcase	30	CLO4, CLO5	Team project + individual viva
Total	100		

Final Group Project

Teams of three are formed in Week 8. The final project integrates the React, routing, ShadCN/ui, Firebase Authentication, Firestore and deployment skills taught in the course. It should extend the students' existing knowledge rather than introduce a new technology stack.

- At least three routes.
- Authentication with protected routes.
- One Firestore collection with basic read/write.
- At least one meaningful ShadCN/ui feature.
- Responsive design and deployment.
- README with setup and feature information.
- Each member owns at least one feature end-to-end.
- Each member works on an individual branch and opens a pull request.
- Each member makes at least eight meaningful commits.
- Each member completes an individual viva.

Topics Intentionally Retained as Core

Area	**Reason**
HTML + semantic structure	Foundation for every web interface.
CSS + Flexbox + Grid + responsive design	Essential for employable front-end layout skills.
JavaScript + DOM + events	Core bridge from static pages to interactive applications.
Promises + async/await + fetch + loading/error states	Required for real API-driven front ends.
Vite + React components/props/state/useEffect	Modern application development foundation.
React Router	Required for realistic multi-page-feeling SPAs and the final project.
ShadCN/ui	Retained as requested; taught as a practical subset.
Firebase Authentication	Retained as a practical authentication skill.
Firestore	Retained at minimum practical scope.
Git/GitHub	Retained at repository/commit/branch/pull-request level.
Deployment	Students should finish the course able to publish working applications.
Six mini projects	Retained to provide frequent individual evidence and progressive practice.

Course Delivery and Teaching–Learning Strategy

- **Laboratory demonstration**: explain only the concepts required for the day's implementation.
- **Live build walkthrough**: instructor constructs a working pattern from an empty or prepared project.
- **Code-along**: students reproduce the core pattern and independently modify part of it.
- **Guided practice**: students solve a small variation while the instructor circulates and diagnoses problems.
- **DevTools practice**: students inspect DOM, styles, network requests and application behavior.
- **Studio critique**: students discuss implementation decisions and common mistakes.
- **Dedicated project evaluation**: one complete class is reserved for each mini-project checkoff.
- **Final integration studio**: students receive structured time to debug and polish the capstone.

Standard 50-Minute Teaching Pattern

Time	Activity
0–5 min	Recall: 2–3 questions and today's outcome
5–15 min	Concept: only the concepts required for today's implementation
15–30 min	Live build: instructor codes the core pattern
30–43 min	Student build: reproduce and modify independently
43–48 min	Debug/review: common mistakes and fixes
48–50 min	Exit check: one conceptual and one practical question

Exception — Project Evaluation Classes
C4 of Weeks 3, 5, 7, 10, 13 and 14 follows the dedicated evaluation procedure rather than the normal 50-minute teaching pattern. This is intentional because the six projects are individual assessments and the course has many students.

Course Boundaries

- The course prepares students for practical front-end development; it does not attempt to make them specialists in UI/UX, testing, databases, mapping, analytics, advanced React architecture, or advanced Git workflows.
- Server-side programming, REST implementation, deep database integration and backend architecture remain outside the core scope and should be emphasized in later Web Programming courses.
- Students may use AI tools openly, but project evaluation requires repository evidence and a short live demonstration/modification to verify ownership and understanding.

Core Graduate Capability After CSE-2340

A student completing the course should be able to design and implement a responsive front-end application, use HTML/CSS and modern JavaScript, manipulate the DOM, consume asynchronous APIs, build React single-page applications with routing and state, use ShadCN/ui, implement Firebase Authentication and basic Firestore persistence, use Git/GitHub for repository-based collaboration, and deploy the resulting application.

Dependency with Later Courses

CSE-2340 provides the front-end foundation without consuming the server-side and database depth required by later courses. CSE-3532 Web Programming can therefore emphasize server-side programming, REST design, database integration, validation and project architecture rather than repeating the full front-end sequence.

Recommended Learning Materials

Resource	Use
MDN Web Docs	Primary reference for HTML, CSS, JavaScript, DOM and browser APIs.
web.dev — Learn CSS / Learn Forms	Practical CSS, responsive and form guidance.
javascript.info	JavaScript fundamentals and browser-side programming.
Eloquent JavaScript	Supplementary JavaScript reference.
React Documentation	Components, state, effects and React patterns.
Vite Documentation	Modern React development environment.
React Router Documentation	Routing and navigation.
Tailwind CSS Documentation	Utility-first responsive styling.
ShadCN/ui Documentation	Practical component usage.
Firebase Documentation	Authentication, Firestore and deployment.
Git / GitHub Documentation	Repositories, commits, branches and pull requests.
Pro Git, 2nd Edition	Supplementary Git reference.
Figma / Pixso Documentation	Basic design handoff and asset inspection.

Revision Summary — V3.4 → V3.5

Change	V3.5 Decision	Expected Benefit
Mini-project evaluation	One full 50-minute C4 evaluation class for each of six projects; MP3 moved to Week 8 C4 because Week 7 C3–C4 is the mid-term	Preserves meaningful individual verification for all six projects while keeping the 100-minute mid-term inside the normal weekly schedule.
Mid-term	Separate 100-minute practical slot; does not consume Week 7 C1–C4	Provides enough uninterrupted time for a realistic practical task.
Figma/Pixso	One focused practical class instead of broader design coverage	Preserves design-reading ability without turning the course into UI/UX training.
JavaScript theory	Reduced language-runtime/detail-heavy coverage	More time for DOM, APIs and React.
React theory	Reduced virtual-DOM/hooks/refs theory	More implementation time without losing essential React skills.
Project-specific libraries	Removed from core syllabus	Reduces technology overload and preserves focus.
Firestore	Minimum practical scope	Prevents database theory from consuming front-end development time.

Git	Repository/commit/branch/PR fundamentals only	Enough for project collaboration without advanced merge workflows.
Evaluation time	Converted from teaching time on project due weeks	Makes the assessment model realistic for a large class.

Assessment Notes

- The six mini projects are individual and progressive.
- Each mini project is directly preceded by the relevant live build or guided practice.
- Each mini project has a dedicated evaluation class.
- Evaluation may use a deployed URL, repository link, rubric, short demonstration and one small live modification.
- AI tools may be used openly; students must be able to explain and modify their submitted work.
- The final group project is team-based, but individual ownership is verified through branches, pull requests, commits and viva.
- The mid-term is a 100-minute practical examination occupying the Week 7 C3–C4 block.

Key Consistency Checks

- Context API is completely absent from the revised syllabus.
- GitHub merge and merge-conflict resolution are completely absent.
- All six mini projects remain and retain their original 40-mark total.
- ShadCN/ui remains included.
- Firebase Authentication remains included.
- Firestore remains included at reduced practical scope.
- The course still has four 50-minute laboratory classes per week.
- The six mini-projects each have a dedicated evaluation class. MP3 is evaluated in Week 8 C4 because Week 7 C3–C4 is reserved for the 100-minute mid-term.
- The mid-term is a 100-minute practical examination occupying the Week 7 C3–C4 block.
- No project evaluation class is counted as new assessment marks; it is the delivery mechanism for the existing 40 continuous-assessment marks.
- The final group project remains 30 marks and the total remains 100.